

```

# You can use the "input" method to get a response from the user.
name = input('What is your name? ')
print('Hi', name, ':)')

What is your name? Daniel

Hi Daniel :)

# Here is a representation of an example Tic-Tac-Toe board using a
# list of lists
#
# The board is:
#      x | o | x
#      -----
#      | o |
#      -----
#      o | x |
exampleBoard = [['x', 'o', 'x'],
                [None, 'o', None],
                ['o', 'x', None]]

# A simple method to print out a Tic-Tac-Toe board
def printBoard(board):
    for row in board:
        print(row)

printBoard(exampleBoard)

['x', 'o', 'x']
[None, 'o', None]
['o', 'x', None]

# Helper methods to pretty-print a Tic-Tac-Toe board

# Returns:
# (1) Space (' ') if the item is None.
# (2) The item itself otherwise.
def prettifyItem(item):
    if item == None:
        return ' '
    else:
        return item

# Prints out a prettified version of the given row
def prettyPrintRow(row):
    prettifiedRow = []
    for item in row:
        prettifiedRow += [prettifyItem(item)]
    print(' ', prettifiedRow[0], '|', prettifiedRow[1], '|',
prettifiedRow[2])

```

Prints out a prettified version of the given board

```
def prettyPrintBoard(board):  
    prettyPrintRow(board[0])  
    print(' -----')  
    prettyPrintRow(board[1])  
    print(' -----')  
    prettyPrintRow(board[2])
```

```
prettyPrintBoard(exampleBoard)
```

```
x | o | x  
-----  
  | o |  
-----  
o | x |
```

Returns:

(1) The winner ('x' or 'o') if there is a winner.
(2) 'Tied!' if the game is over and there is no winner.
(3) None otherwise.

```
def gameWinner(board):  
    for i in range(3):  
        # Check row i:  
        if (board[i][0] == board[i][1]) and (board[i][1] == board[i]  
[2]):  
            return board[i][0]  
        # Check column i:  
        if (board[0][i] == board[1][i]) and (board[1][i] == board[2]  
[i]):  
            return board[1][i]  
        # Check the diagonals:  
        if ((board[0][0] == board[1][1]) and (board[1][1] == board[2][2]))  
or (  
            (board[0][2] == board[1][1]) and (board[1][1] == board[2]  
[0]))):  
            return board[1][1]  
        # Check if there are any open spaces. If there are, the game is  
still ongoing.  
        for r in range(3):  
            for c in range(3):  
                if board[r][c] == None:  
                    return None  
        # If there are no open spaces, the game is tied.  
        return 'Tied!'
```

```
print(gameWinner([['o', 'o', 'x'],  
                  ['x', 'x', 'o'],  
                  ['o', 'o', 'x']]))
```

Tied!

```

# A Tic-Tac-Toe player using user input.
def userInputPlayer(board):
    # Get a response from the user in "r,c" format.
    response = input('What is your next move (in "r,c" format)? ')
    return [int(response[0]), int(response[2])]

userInputPlayer(None)

What is your next move (in "r,c" format)? 1,1

[1, 1]

# A Tic-Tac-Toe player that moves to the first open space.
def sillyPlayer(board):
    for r in range(3):
        for c in range(3):
            if board[r][c] == None:
                return [r, c]

def symbolForPlayer(playerIndex):
    if playerIndex == 0:
        return 'x'
    else:
        # Here, playerIndex is 1.
        return 'o'

# A game engine to that takes two Tic-Tac-Toe players and runs a full
game.
#
# Returns:
# (1) The winner ('x' or 'o') if there is a winner.
# (2) 'Tied!' if there is no winner.
def ticTacToeEngine(player1, player2):
    players = [player1, player2]
    nextPlayerIndex = 0
    # Initialize the board, no moves have been made yet.
    board = [[None] * 3, [None] * 3, [None] * 3]
    while gameWinner(board) == None:
        nextPlayer = players[nextPlayerIndex]
        move = nextPlayer(board)
        r = move[0]
        c = move[1]
        if board[r][c] != None:
            print('That position is already filled!')
            continue
        board[r][c] = symbolForPlayer(nextPlayerIndex)
        nextPlayerIndex = (nextPlayerIndex + 1) % 2
        prettyPrintBoard(board)
    print('')
    print('')

```

```
return gameWinner(board)
```

```
winner = ticTacToeEngine(userInputPlayer, sillyPlayer)
```

```
print(winner, 'won!')
```

What is your next move (in "r,c" format)? 1,1

```
  |  |  
--  
  | x |  
--  
  |  |
```

```
o |  |  
--  
  | x |  
--  
  |  |
```

What is your next move (in "r,c" format)? 0,1

```
o | x |  
--  
  | x |  
--  
  |  |
```

```
o | x | o  
--  
  | x |  
--  
  |  |
```

What is your next move (in "r,c" format)? 2,0

```
o | x | o  
--  
  | x |  
--  
x |  |
```

```
o | x | o  
--  
o | x |  
--
```

```
x |  |
```

What is your next move (in "r,c" format)? 1,2

```
o | x | o
-----
o | x | x
-----
x |  |
```

```
o | x | o
-----
o | x | x
-----
x | o |
```

What is your next move (in "r,c" format)? 2,1

That position is already filled!

What is your next move (in "r,c" format)? 3,3

```
-----
-----
IndexError                                Traceback (most recent call
last)
Cell In[15], line 33
      30     print('')
      31     return gameWinner(board)
--> 33 winner = ticTacToeEngine(userInputPlayer, sillyPlayer)
      34 print(winner, 'won!')
```

```
Cell In[15], line 23, in ticTacToeEngine(player1, player2)
      21 r = move[0]
      22 c = move[1]
--> 23 if board[r][c] != None:
      24     print('That position is already filled!')
      25     continue

IndexError: list index out of range
```